

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Майстренко Анастасия Николаевна

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Опираясь на технический дизайн микросервисной архитектуры из ДЗ4, реализовать разделение монолитного бэкенд-приложения сервиса аренды недвижимости на отдельные микросервисы с собственными базами данных (database-per-service) и межсервисным взаимодействием.

Ход работы

1. Состав реализованных сервисов

Монолит из ЛР1 разделён на четыре микросервиса и API Gateway согласно проекту ДЗ4. Каждый сервис — самостоятельное приложение Express + TypeORM + routing-controllers со своей базой данных и своим портом (таблица 1).

Таблица 1 — Реализованные сервисы

Сервис	Порт	База данных	Сущности / функции
API Gateway	8000	—	Маршрутизация /api/v1 на сервисы
Identity	8001	users_db	User: регистрация, вход, профиль
Catalog	8002	catalog_db	Property, Amenity: каталог, поиск
Booking	8003	booking_db	Booking, Review: сделки, отзывы
Messaging	8004	messaging_db	Conversation, Message: переписка

2. Разделение базы данных (database-per-service)

Каждый сервис владеет собственной БД и не имеет доступа к чужим таблицам. Внешние ключи между сервисами отсутствуют — хранятся только идентификаторы (например, Property.ownerId, Booking.propertyId). Для устранения зависимости от чужих данных применена денормализация: Booking хранит снимок названия объекта (propertyTitle), полученный из Catalog на момент создания брони. Локально каждая БД — отдельный файл SQLite, в продакшн-конфигурации — отдельная база PostgreSQL (rental_identity, rental_catalog, rental_booking, rental_messaging).

Изоляция подтверждена: в каждой БД присутствуют только её собственные таблицы — users_db: user; catalog_db: property, amenity, property_amenity; booking_db: booking, review; messaging_db: conversation, message.

3. Межсервисное взаимодействие

Реализованы синхронные вызовы по REST через внутренние (/internal) эндпоинты, защищённые служебным заголовком X-Internal-Token. Аутентификация пользователя — JWT с общим секретом, который каждый сервис проверяет локально (stateless), без обращения к Identity на каждый запрос. Ключевой пример взаимодействия — создание бронирования:

- клиент вызывает POST /api/v1/bookings через Gateway → Booking Service;
- Booking синхронно запрашивает у Catalog GET /internal/properties/{id} и получает цену и владельца;
- Booking рассчитывает стоимость и сохраняет сделку со снимком названия объекта;
- при подтверждении сделки Booking вызывает Catalog PATCH /internal/properties/{id}/status и переводит объект в статус rented (при завершении — обратно в available).

Внутренние эндпоинты не публикуются наружу: Gateway маршрутизирует только публичные пути (/auth, /users, /properties, /amenities, /bookings, /conversations), а /internal остаётся доступным лишь внутри сети сервисов.

4. API Gateway

Шлюз (http-proxy-middleware) принимает все клиентские запросы по префиксу /api/v1 и проксирует их в нужный сервис, срезая префикс. Это даёт клиенту единую точку входа и единый адрес, скрывая внутреннее устройство системы.

```
/api/v1/auth, /api/v1/users    -> identity-service:8001
/api/v1/properties, /amenities -> catalog-service:8002
/api/v1/bookings              -> booking-service:8003
/api/v1/conversations          -> messaging-service:8004
```

5. Проверка работоспособности

Все сервисы запущены одновременно, сквозной сценарий выполнен через Gateway. Результаты приведены в таблице 2 — все проверки пройдены.

Таблица 2 — Результаты проверки

Проверка	Результат
Все 5 сервисов отвечают на /health	ok
Регистрация/вход через Gateway → Identity	201 / 200
Проверка JWT в других сервисах (/users/me)	200
Создание объекта и поиск через Gateway → Catalog	201 / 200

Бронирование: Booking → Catalog GET /internal (цена, владелец)	цена 12500 верно
Денормализация: снимок названия объекта в брони	получен из Catalog
Подтверждение брони: Booking → Catalog PATCH статуса	available → rented
Завершение брони: возврат статуса объекта	rented → available
Отзыв по завершённой сделке (Booking)	201
Переписка через Gateway → Messaging	201
/internal недоступен снаружи через Gateway	404
/internal без служебного токена	401
/internal со служебным токеном	200

Изоляция данных подтверждена отдельными файлами БД с непересекающимся набором таблиц.

6. Структура проекта

```

shared/                — общие хелперы (entity-controller, auth,
тоkens,
                        data-source-factory, internal-client)
services/identity/ — модели, dto, контроллеры, app.ts, data-
source
services/catalog/  — то же
services/booking/  — то же (вызывает Catalog)
services/messaging/ — то же
gateway/app.ts      — API Gateway (http-proxy-middleware)
docker-compose.yml + init-dbs.sql — PostgreSQL с 4 БД

```

Вывод

Монолитное приложение из ЛР1 разделено на четыре микросервиса (Identity, Catalog, Booking, Messaging) и API Gateway в соответствии с проектом ДЗ4. Реализован принцип database-per-service: каждый сервис владеет собственной БД, связи между сервисами выражены идентификаторами и денормализацией, а недостающие данные запрашиваются по защищённым внутренним REST-эндпоинтам.

Межсервисное взаимодействие продемонстрировано на сценарии бронирования, где Booking Service синхронно обращается к Catalog Service за ценой и владельцем объекта и обновляет его статус. Все сервисы запускаются совместно, публичный API доступен через единый шлюз, работоспособность подтверждена сквозным сценарием. Архитектура обеспечивает слабую связанность, независимое развёртывание и масштабирование сервисов.